Typesec

covers typesec (0.4.0)

Type-Level Security for Agentic AI

Alexy Khrabrov

chiefscientist.org

&

Codex with ChatGPT 5.5

Contents

1 Preface

This book describes Typesec, a system for carrying authorization evidence in types while leaving policy decisions explicit at runtime. It is both a design record and a codebase guide. The intent is not just to say what the repository contains, but to explain why each piece exists, how the pieces fit together, and what security property the system is trying to make harder to accidentally lose.

One direct inspiration was David Andrzejewski's Scale By the Bay 2018 talk, "Privacy aware data science in Scala with monads and type level programming". That talk connected notebook-era data science, information-flow control, and type-level programming. It also pointed back to the Haskell SecLib model of wrapping data in a security-labeled container. Typesec takes that lineage in a Rust direction: runtime policy still decides, but the proof and the protected data shape are carried by types. A cleaned transcript of David's talk is included at `docs/david-andrzejewski-scale-by-the-bay-2018-transcript.md`.

The project began from a simple observation: agentic software makes ordinary authorization mistakes more dangerous. A human-facing web request usually has a short life. An AI agent can be long-running, can call many tools, can make many intermediate decisions, and can wander through code paths that were not anticipated when the first permission guard was written. If security depends on remembering to call `check_permission()` before every dangerous action, the system is only as strong as its most forgetful call site.

Typesec moves the central permission proof into Rust's type system. Runtime policy still matters. RBAC and ODRL policies are parsed and evaluated at runtime. But once a policy engine allows an action, the result is not just a boolean. The result is an unforgeable typed value:

`Capability<CanWrite, Report>`

That value becomes the proof required by sensitive APIs. Code that writes a report should accept a write capability. Code that reads sensitive data should accept a sensitive-read capability. Code that exfiltrates data should be forced to say so in its type signature. The ideal is a codebase where the compiler can show us the security boundary before the program ever runs.

The newer `SecureValue<L, T, R>` layer extends that idea from operations to data itself. A sensitive string can be transformed while it remains opaque. It can be combined with other values while the type-level label tracks the more restrictive result. It cannot be unwrapped or declassified unless the caller has the corresponding capability.

The repository now lives at:

`git@github.com:querygraph/typesec.git`

The local branch `main` tracks `origin/main`. Before publishing, the workspace was checked with:

```
cargo check --workspace
```

The Grust integration was also checked with:

```
cargo check --example company_graph_grust_sail
```

2 The Problem

Most authorization code is guard-based. A function starts with a condition:

```
if acl.check(user, "write", resource) {
    resource.write(data);
}
```

This pattern is familiar, easy to understand, and often good enough for small systems. It also has a structural weakness: the guard is separate from the operation. Every call path has to remember the guard. Every refactor has to preserve the guard. Every new integration has to know which guard applies.

Agentic systems stress this weakness. An agent may plan, retrieve data, call a tool, revise the plan, call another tool, ask another model, and then publish a result. Each of those steps can cross a policy boundary. Some boundaries are ordinary application boundaries, such as read versus write. Others are AI-specific boundaries, such as train, infer, or exfiltrate.

The central design question for Typesec is:

> Can we make the permission proof part of the API shape, so the dangerous action is not callable unless the proof is already in scope?

Typesec does not remove runtime policy evaluation. It changes what runtime policy evaluation returns. A policy check that allows an action mints a capability. The capability is then consumed or borrowed by APIs that require that access.

The most important consequence is that permission becomes visible in function signatures:

```
fn write_report(cap: &Capability<CanWrite, Report>, report: &Report) {
    // The body can assume the policy engine approved this subject
    // for this permission and this resource.
}
```

There is no matching function that takes only `&Report` and quietly writes it. If such a function appears, it is visible during review as a policy bypass.

3 The Security Model

Typesec is built around a small set of invariants.

First, capabilities are unforgeable outside `typesec-core`. The `Capability<P, R>` fields are private, and the constructor that skips policy evaluation is `pub(crate)`. External crates cannot write:

```
Capability { ... }
```

and they cannot call the unchecked constructor. They must ask a policy engine.

Second, permission types are sealed. The `Permission` trait is implemented by Typesec's own marker types, such as `CanRead`, `CanWrite`, `CanDelete`, `CanReadSensitive`, `AiCanInfer`, `AiCanTrain`, and `AiCanExfiltrate`. External crates cannot invent `CanDoAnything` and use it as a back door.

Third, resource types implement a `Resource` trait. A resource exposes a stable identifier, and that identifier is what runtime policy engines evaluate. The type parameter still matters. `Capability<CanWrite, Report>` is not the same type as `Capability<CanWrite, CustomerRecord>`, even if both are represented at runtime by strings.

Fourth, agent authentication is represented as typestate. The core agent starts as:

```
Agent<Unauthenticated>
```

After successful authentication it becomes:

```
Agent<Authenticated>
```

Only the authenticated state exposes capability-requesting behavior. This makes "forgot to authenticate" a type error instead of a late runtime surprise.

Fifth, every policy decision is auditable. The runtime bridge emits structured events through `tracing`, so production users can connect the policy layer to their logging or SIEM infrastructure.

These invariants are intentionally modest. Typesec is not pretending that the type system can know every business rule at compile time. The type system enforces that sensitive operations require a proof. Runtime policy decides whether the proof should be minted.

4 Workspace Tour

The repository is a Rust workspace with nine crates:

```
typesec           facade crate re-exporting the common API
typesec-core      traits, phantom types, Capability, SecureValue, PolicyEngine,
typestate
typesec-rbac      YAML RBAC parser, validator, engine, and code generator
typesec-odrl      YAML ODRL subset, constraints, prohibitions, and audit events
typesec-agent     SecureAgent wrapper for async capability requests and execute
typesec-integrations JWT/OIDC, WorkOS FGA, and Arcade-style tool auth engines
typesec-macro     derive and policy macros for typed role declarations
typesec-cli       validate, check, generate, and run commands
typesec-python    PyO3 bindings for Rust-backed Python policy gates
```

The repository also includes:

```
examples/rbac_agent.rs
examples/odrl_agent.rs
examples/provider_integrations.rs
examples/company_graph/company_graph_grust_sail.rs
examples/company_graph/langchain_company_graph.py
policies/rbac-example.yaml
policies/odrl-example.yaml
docs/company-graph-grust-sail.md
docs/david-andrzejewski-scale-by-the-bay-2018-transcript.md
docs/improvements.md
docs/oauth-provider-integrations.md
docs/typesec-and-auth-frameworks.md
docs/typesec-claude1.md
tests/python/test_cli_policy.py
```

The root workspace uses Rust 2024. Common dependencies are declared in `[workspace.dependencies]`: `tokio`, `serde`, `serde_yaml`, `serde_json`, `clap`, `tracing`, `thiserror`, `anyhow`, `syn`, `quote`, `proc-macro2`, and `glob`, `jsonwebtoken`, and `reqwest`.

One important dependency change happened late in the day. The local example had originally depended on a path checkout of Grust:

```
grust = { path = "../../../grust/crates/grust", features = ["sail"] }
```

After the Grust crates were published, the dependency was moved to the published facade crate. The current examples use Grust 0.4 typed graph and typed backend support:

```
[dev-dependencies]
grust-graph = { version = "0.4.0", features = ["typed-zod-rs", "sail"] }
```

The package is named `grust-graph`, while its library is imported as `grust`. The facade re-exports the core graph API and, with the `sail` feature enabled, the Sail adapter types. The `typed-zod-rs` feature lets Typesec route policy YAML and JSON through Zod schemas before typed Rust structs lower into a Grust graph. The same graph schema is then passed to Grust backends with `put_typed_graph`, so persistence validates the graph shape instead of accepting a loose property graph.

5 `typesec-core`

The core crate is where the security idea becomes concrete. It contains the unforgeable capability type, permission markers, resource abstraction, policy engine trait, typestate agent, policy combinators, and security-labeled values.

5.1 Capabilities

The central type is:

```
pub struct Capability<P: Permission, R: Resource> {
    subject: String,
    resource_id: String,
    _permission: PhantomData<fn() -> P>,
    _resource: PhantomData<fn() -> R>,
}
```

At runtime, the value stores the subject and resource identifier. The permission and resource types are represented with `PhantomData`. They cost nothing at runtime, but they force the compiler to distinguish a read capability from a write capability.

The constructor is deliberately hidden:

```
pub(crate) fn new_unchecked(
    subject: impl Into<String>,
    resource_id: impl Into<String>,
) -> Self
```

That one visibility choice carries much of the design. The capability can be minted by code inside `typesec-core`, but not by consumers. Consumers call `mint_capability`, which first asks a `PolicyEngine`.

Capabilities are intentionally not `Clone`. A privilege should not spread through the program by accident. If code needs to share a capability, it passes a reference. This is a small choice, but it aligns the ergonomics with the security model: possession of a capability is meaningful.

5.2 Permissions

Permissions are marker types. The names are runtime strings when a policy engine needs to evaluate a request, but static types when APIs require proof. The project includes ordinary permissions:

```
read
write
delete
execute
delegate
read_sensitive
write_sensitive
declassify
```

It also includes AI-oriented permissions:

```
ai:infer
ai:train
ai:exfiltrate
```

The exfiltration permission is especially important. If an agent can send data outside a trust boundary, that path should not be hidden inside an ordinary "write" operation. A function that requires `Capability<AiCanExfiltrate, CustomerData>` announces the risk in its signature.

The declassification permission is similarly explicit. If code lowers a protected value from `Sensitive` or `Secret` to `Public`, it should not look like an ordinary read. A function that accepts `Capability<CanDeclassify, Report>` is telling reviewers that this path intentionally releases a derived value.

5.3 Secure Values

SecLib's useful lesson for Typesec is that the protected value itself should have a shape the compiler can recognize. Typesec now has:

```
pub struct SecureValue<L: PrivacyLevel, T, R: Resource> {
    value: T,
    resource_id: String,
    _label: PhantomData<fn() -> L>,
    _resource: PhantomData<fn() -> R>,
}
```

The fields are private. Consumers cannot extract `value` by pattern matching. They can transform it:

```rust
let email: SecureValue<Sensitive, String, CustomerRecord> =
    SecureValue::protect(customer.email, &customer);

let domain = email.map(|addr| addr.split('@').last().unwrap_or("").to_owned());
```

The label is preserved through `map`. When two protected values are combined with `zip`, the `Join` trait computes the more restrictive privacy label:

```
Public + Sensitive -> Sensitive
Sensitive + Secret -> Secret
Internal + Public -> Internal
```

This is deliberately small. It is not a complete information-flow-control language, and it does not model every SecLib operation. But it gives application code a first-class way to keep sensitive data opaque while still doing useful work with it.

Extraction is limited to explicit release paths. `SecureValue<Public, T, R>` can be unwrapped with `into_public`. Any protected label can be revealed with a `Capability<CanReadSensitive, R>`. To lower the label, code must hold:

```
Capability<CanDeclassify, R>
```

That makes declassification visible at the call site and in the policy log.

5.4 Resources

Resources implement:

```rust
pub trait Resource {
    fn resource_id(&self) -> &str;
    fn resource_type() -> &'static str;
}
```

The policy engine sees the identifier. Rust sees the type. This lets Typesec bridge dynamic policy files with typed application code.

For quick CLI and test paths, the core crate also provides generic resources. Examples define domain-specific resources, such as employee nodes, relationships, company graphs, and employee networks.

5.5 Policy Results

All policy engines return:

```rust
pub enum PolicyResult {
    Allow,
    Deny(String),
    Delegate(String),
}
```

`Allow` mints a capability. `Deny` returns an error with a reason. `Delegate` means this engine has no definitive answer and another engine may decide. ODRL uses delegation naturally: if an ODRL policy has no matching rule, it can defer to RBAC.

The error type for capability acquisition is:

```
pub enum CapabilityError {
    Denied { reason: String },
    UnhandledDelegation,
    EngineError(String),
}
```

One future improvement is to make delegation more first-class at the capability-minting boundary. Today, if a single engine delegates and no wrapper handles that delegation, minting fails with `UnhandledDelegation`. The composition layer solves this for deployed combinations, but the distinction is worth documenting.

5.6 Minting

The minting flow is the core runtime bridge:

```
agent.request_capability::<CanWrite, Report>(&report)
  -> engine.check(subject, "write", report.resource_id())
  -> PolicyResult::Allow
  -> Capability::new_unchecked(subject, resource_id)
```

The function that performs this is `mint_capability`. It emits an audit event for every decision and only calls the unchecked constructor after an allow.

5.7 Typestate

The typestate module defines:

```
Agent<Unauthenticated>
Agent<Authenticated>
```

The state marker trait is sealed. External crates cannot add fake states. The only transition from unauthenticated to authenticated is through `authenticate`, which consumes the unauthenticated agent and returns an authenticated one.

The authentication implementation is intentionally small. It checks that the subject and token are present. A production system would verify a JWT, API key, or identity-provider token. The point of this crate is not to own identity. The point is to make the authenticated state visible in the type system after identity has been established.

5.8 Combinators

The combinator module lets multiple policy engines be combined. It supports four strategies:

```
PriorityOrder   first non-delegating answer wins
AllowIfAll      all definitive engines must allow
AllowIfAny      any allow is enough unless all deny or delegate
DenyOverrides   any deny beats any allow
```

`DenyOverrides` is the conservative XACML-style choice. `PriorityOrder` is useful when a more specific policy should get the first chance to decide and fall back to a broader one only on delegation.

6 OAuth, Arcade, WorkOS, and Typesec

Typesec is not an OAuth replacement. That point matters because the modern agent-security stack already has strong identity and delegation systems. OAuth, OIDC, AuthKit, WorkOS, Arcade, and provider-specific consent flows all solve real problems that Typesec should not try to reimplement.

The better architecture is layered:

```
OAuth/OIDC proves identity and delegates external authority
WorkOS models enterprise users, organizations, sessions, RBAC, and FGA
Arcade manages agent-facing OAuth/tool authorization for SaaS tools
Typesec converts allowed decisions into typed local capabilities
```

This comparison is also recorded in `docs/typesec-and-auth-frameworks.md`. The short version is:

> OAuth proves and delegates identity. Typesec turns authorization decisions into compile-time-visible authority inside agent/tool code.

6.1 The Comparison

The systems operate at different layers:

```
Typesec   code-level enforcement for agent/tool execution
Arcade    MCP/tool runtime and delegated user auth for external services
WorkOS    identity, OAuth apps, RBAC/FGA, enterprise auth infrastructure
```

Typesec's core primitive is `Capability<P, R>` plus typestate `Agent<Authenticated>`. Arcade's core runtime primitive is a user-specific tool authorization and managed OAuth/API-key token state. WorkOS's primitives are OAuth/OIDC tokens, organization claims, roles, permissions, and FGA access checks.

That gives each system a natural strength:

```
Use WorkOS when you need enterprise login, SSO, organizations, RBAC, IdP sync,
and resource-scoped FGA for application objects.

Use Arcade when an agent needs to act as a user inside external SaaS systems
such as Gmail, Slack, GitHub, Jira, or Google Drive.

Use Typesec when local tool code must not run unless the program holds a typed
proof that authorization already happened.
```

Arcade and WorkOS still return runtime facts. A token verifies. A consent flow completes. An authorization API returns `authorized: true`. Those facts are important, but ordinary application code can still ignore them unless every call site is disciplined. Typesec adds the local last mile: the handler is written to require a typed capability, so the call cannot be made by accident.

6.2 What Arcade Does

Arcade is closest to the agent-tool problem. It handles OAuth 2.0, API keys, and user tokens for tools. In an agent setting, that means a user can authorize Gmail once, Arcade can store and

refresh the provider tokens, and an agent can later call a Gmail tool on that user's behalf without the model seeing secrets.

Arcade also separates two boundaries:

```
resource-server auth  protects access to the MCP server or gateway
tool-level auth       authorizes the specific third-party tool call
```

That distinction maps cleanly to Typesec. Arcade can answer:

```
Has user@example.com authorized Gmail.ListEmails?
```

Typesec can then mint:

```
Capability<CanExecute, GenericResource>
```

where the resource id is `Gmail.ListEmails` or a local alias such as `gmail/list`.

The repository's `ArcadeToolAuthEngine` is intentionally small. It maps a local resource id to an Arcade tool name, asks the configured Arcade endpoint whether the user's authorization is complete, and returns `PolicyResult::Allow` only when the provider response is complete. Pending authorization is a denial with the provider URL included when available.

```
let arcade = ArcadeToolAuthEngine::new(arcade_api_key)
    .with_tool_mapping("gmail/list", "Gmail.ListEmails");

let gmail = GenericResource::new("gmail/list", "tool");
let cap: Capability<CanExecute, GenericResource> =
    agent.request_capability(&gmail).await?;
```

The important design choice is that Arcade remains the OAuth/tool runtime, while Typesec remains the local proof system. Arcade knows how to complete OAuth. Typesec knows how to make the local tool handler require proof.

6.3 What WorkOS Does

WorkOS is broader enterprise auth infrastructure. AuthKit and Connect cover login, OAuth applications, SSO, user sessions, organizations, token issuance, and token verification. WorkOS Fine-Grained Authorization extends RBAC into a hierarchical, resource-scoped model for application objects such as organizations, workspaces, projects, and apps.

For Typesec, the useful WorkOS split is:

```
JWT claims     fast checks for org-wide roles and permissions
FGA API        precise checks for a specific action on a specific resource
```

That split fits Typesec's existing policy-composition model. The token can be verified once and converted into a `JwtClaimsEngine`. The JWT engine can allow obvious org-wide permissions and delegate everything else. The WorkOS engine can then call the Authorization API for resource-specific checks.

```
let jwt_engine = Arc::new(JwtClaimsEngine::from_permissions(
    "user@example.com",
```

```
    ["org:view".to_string()],
));

let workos_engine = Arc::new(WorkOsFgaEngine::new(workos_api_key));

let engine = PolicyEngineBuilder::new()
    .add_engine(jwt_engine)
    .add_engine(workos_engine)
    .strategy(CombineStrategy::PriorityOrder)
    .build();
```

Typesec resource ids use a simple convention for WorkOS FGA:

```
project/proj_123
workspace/ws_123
```

`WorkOsFgaEngine` parses the prefix as the resource type slug and the suffix as the external resource id. A `write` capability request on `project/proj_123` becomes a WorkOS-style permission check for `project:write` on that resource.

6.4 The Integrative Architecture

The `typesec-integrations` crate puts these ideas behind the same `PolicyEngine` trait used by RBAC, ODRL, and graph policies.

```
OIDC/AuthKit access token
  -> JwtAuthenticator verifies signature, issuer, audience, expiry
  -> VerifiedSubject becomes the authenticated Typesec subject

Capability request
  -> JwtClaimsEngine handles fast org-wide permissions
  -> WorkOsFgaEngine handles app resource permissions
  -> ArcadeToolAuthEngine handles external SaaS tool authorization
  -> PolicyResult::Allow
  -> Capability<P, R>
  -> ProtectedTool<P, R, _> can run
```

Representative setup from `examples/provider_integrations.rs` looks like this:

```
let engine = PolicyEngineBuilder::new()
    .add_engine(Arc::new(JwtClaimsEngine::from_permissions(
        "user@example.com",
        ["read".to_string()],
    )))
    .add_engine(Arc::new(WorkOsFgaEngine::new(workos_api_key)))
    .add_engine(Arc::new(
        ArcadeToolAuthEngine::new(arcade_api_key)
            .with_tool_mapping("gmail/list", "Gmail.ListEmails"),
    ))
    .strategy(CombineStrategy::PriorityOrder)
    .build();
```

Then local tool code can be shaped around the capability:

```
fn gmail_list(_resource: &GenericResource) -> ToolFuture<'_> {
    Box::pin(async {
        // Call the actual MCP/Arcade/Gmail implementation here.
        Ok(())
    })
}

let resource = GenericResource::new("gmail/list", "tool");
let tool = ProtectedTool::<CanExecute, _, _>::new(
    "gmail.list",
    "List email messages",
    resource,
    gmail_list,
);

let cap: Capability<CanExecute, GenericResource> =
    agent.request_capability(&GenericResource::new("gmail/list", "tool")).await?;

tool.invoke(&agent, &cap).await?;
```

This is the central integration claim. WorkOS can say who the user is and what app resource they can access. Arcade can say whether the user authorized the external SaaS tool. Typesec can make the local implementation impossible to invoke without carrying that authorization as a typed proof.

7 `typesec-agent`

`typesec-core` stays dependency-light. It does not need `tokio`. The `typesec-agent` crate wraps the core typestate agent in a higher-level `SecureAgent` that exposes async capability requests and execution.

The unauthenticated constructor is:

```
let agent = SecureAgent::new(Arc::new(engine));
```

Authentication returns a new type:

```
let agent = agent.authenticate(Credentials::new("agent:bot", "token"))?;
```

Only `SecureAgent<Authenticated>` has:

```
request_capability::<P, R>(&self, resource: &R)
execute(&self, cap: &Capability<P, R>, resource: &R, action)
```

The `execute` method captures the project's main ergonomic pattern. It takes a capability reference and a resource reference, logs the execution, and then runs the action. The newer `ProtectedTool` wrapper applies the same idea to agent-tool and MCP-style handlers: the tool advertises a required permission and resource, and `invoke` requires the matching `Capability<P, R>`.

```
let tool = ProtectedTool::<CanExecute, _, _>::new(
    "gmail.list",
    "List Gmail messages",
    GenericResource::new("gmail/list", "tool"),
    gmail_list,
);

tool.invoke(&agent, &execute_cap).await?;
```

The underlying `execute` method runs an async closure. The closure cannot be reached through this method without a capability.

8 `typesec-integrations`

The integrations crate is intentionally outside `typesec-core`. Provider adapters need HTTP clients, JWT validation, provider-specific endpoints, and credential handling. The core crate should not own those dependencies.

The crate currently contains four modules:

```
http     small injectable JSON HTTP client abstraction
jwt      OIDC/JWT verification and JWT-claims policy engine
workos   WorkOS FGA policy engine
arcade   Arcade-style tool authorization policy engine
```

The `http` module provides `ReqwestHttpClient` for production use plus `StaticHttpClient` and `RecordingHttpClient` for tests and examples. This lets the example exercise provider-like behavior without real credentials:

```
let workos_http = StaticHttpClient::new().with_response(
    "https://api.workos.test/authorization/organization_memberships/user@example.
com/check",
    json!({ "authorized": true }),
);
```

The JWT module has two layers. `JwtAuthenticator` validates a token against a JWKS endpoint, issuer, audience, algorithm list, and expiry. Once a token is verified, `VerifiedSubject` carries the subject, organization id, membership id, role, and permissions. `JwtClaimsEngine` then acts as a fast policy engine for permissions already embedded in the token.

The WorkOS module translates a Typesec resource id into a WorkOS FGA check. For example:

```
resource: project/proj_123
action:   write
check:    project:write on project/proj_123
```

The Arcade module handles external tool authorization. It maps local resource ids to Arcade tool names and only allows execution when the provider reports a completed authorization:

```
let arcade = ArcadeToolAuthEngine::new(arcade_api_key)
    .with_tool_mapping("gmail/list", "Gmail.ListEmails");
```

These engines are not special cases in the capability system. They implement the same `PolicyEngine` trait as RBAC, ODRL, and graph policies. That is the point: external OAuth and enterprise authorization decisions become ordinary inputs to `mint_capability`.

This pattern is small enough to be adopted by application code directly. A database write wrapper can require a write capability. A vector-store retrieval wrapper can require a read capability. An outbound HTTP publisher can require an exfiltration capability.

The crate also includes `AgentBuilder`, which can wrap a single engine or build a composed engine from a primary and fallback. The default composed behavior is priority order: the primary decides unless it delegates.

9 RBAC

The `typesec-rbac` crate implements role-based access control from YAML. A policy has roles and assignments:

```yaml
roles:
  - name: analyst
    permissions: [read, read_sensitive]
    resources: ["reports/*", "metrics/*"]
  - name: admin
    inherits: [analyst]
    permissions: [delete, delegate]
    resources: ["*"]

assignments:
  - subject: "agent:data-pipeline"
    roles: [analyst]
```

The engine compiles this into subject grants. Role inheritance is flattened during construction, not recomputed from scratch at every check. A request then checks:

```
subject -> assigned roles -> effective grants -> permission and glob match
```

The glob matching is intentionally simple. Resource patterns such as `reports/*`, `employee/**`, and `*` are enough for the first version. The code uses the `glob` crate rather than ad hoc string prefixes.

RBAC is the system's practical baseline. It answers common operational questions:

```
Can agent:data-pipeline read reports/q1?
Can agent:deploy-bot write code/main.rs?
Can agent:superuser delete anything?
```

The crate also contains code generation support. `typesec generate` can emit typed Rust structs from an RBAC policy. This is one of the paths toward the larger promise: if a role is renamed in policy, code referring to the old typed role should fail to compile.

10 ODRL

The `typesec-odrl` crate implements a focused subset of the Open Digital Rights Language. ODRL is useful for AI agents because it can express obligations, prohibitions, and contextual constraints.

A policy can say:

```yaml
policies:
  - uid: "policy:ai-agent-001"
    type: Set
    rules:
      - type: permission
        assignee: "agent:summarizer"
        action: read
        target: "asset:customer-data"
        constraints:
          - leftOperand: purpose
            operator: eq
            rightOperand: "analytics"
      - type: prohibition
        assignee: "agent:summarizer"
        action: exfiltrate
        target: "asset:customer-data"
```

This says the summarizer can read customer data for analytics, but cannot exfiltrate it. That distinction matters for agents because the same underlying data may be safe to summarize internally and unsafe to send outside the system.

The ODRL engine evaluates subject, action, target, and constraints. Constraints are evaluated against a `ConstraintContext`, which can include purpose, date-time, and custom key values. The CLI exposes `--purpose` for the common case.

Conflict resolution is conservative:

```
prohibition beats permission
```

If a matching prohibition passes its constraints, the result is a deny. If a permission matches and no prohibition overrides it, the result is allow. If no rule applies, the result is delegate.

The crate also emits structured ODRL audit events. These include the policy UID, matched rule type, subject, action, target, verdict, and constraint evaluation results. This gives operators a reason trail, not just a final boolean.

11 Macros

The `typesec-macro` crate sketches two developer-experience paths.

The first is a derive macro:

```rust
#[derive(TypesecRole)]
#[role(permissions = "read,write", resources = "code/*")]
pub struct Engineer;
```

The second is an inline policy macro:

```
policy! {
    role Analyst {
        can [read, read_sensitive] on ["reports/*", "metrics/*"];
    }
}
```

Macros are not the security core. The core is capabilities and policy-engine minting. The macros are there to make typed policy declarations less tedious and to give future generated code a natural shape.

12 CLI

The CLI is the bridge for humans, scripts, and non-Rust agents. It has four commands:

```
validate   parse and validate a policy YAML file
check      evaluate one subject/action/resource query
generate   emit typed Rust code from an RBAC policy
run        simulate agent execution under policy
```

The `check` command is especially important because it makes Typesec usable from Python or shell without native bindings:

```
cargo run -q -p typesec-cli -- check \
  --policy policies/rbac-example.yaml \
  --subject agent:data-pipeline \
  --action read \
  --resource reports/q1
```

An allow exits successfully and prints an allow result. A deny exits with status `1`. A delegate exits with status `2`. That makes the command a simple policy oracle for external tools.

The format is detected from YAML shape unless `--format rbac` or `--format odrl` is passed explicitly. ODRL checks can receive a purpose:

```
cargo run -q -p typesec-cli -- check \
  --policy policies/odrl-example.yaml \
  --subject agent:summarizer \
  --action read \
  --resource customer-data \
  --purpose analytics
```

One improvement noted in `docs/improvements.md` is to add a stable machine-readable mode, such as `typesec check --json`. Today Python can use exit codes, but parsing stdout is not ideal for long-term integrations.

13 Examples

Typesec includes examples at several layers: pure Rust RBAC, pure Rust ODRL, OAuth-provider integration, and company-graph integrations that show how an agent tool boundary might look in a larger system.

13.1 RBAC Agent

The RBAC example shows an authenticated agent requesting a capability and then executing a protected action. The interesting part is not the business object. The interesting part is the call shape: the protected operation is not called until after policy has minted a capability.

The example also demonstrates denial. An agent assigned to a role with read permissions cannot use a write permission unless the policy grants it.

The updated RBAC example also demonstrates the SecLib-inspired data container. It protects a report summary as `SecureValue<Sensitive, String, GenericResource>`, maps it to a length digest while it remains opaque, denies declassification for the ordinary analyst, and then allows a `privacy_reviewer` role to declassify the digest after policy mints `Capability<CanDeclassify, GenericResource>`.

13.2 ODRL Agent

The ODRL example demonstrates contextual policy. Purpose matters. A read for analytics may be allowed while a read for billing delegates or denies. An exfiltration action can be prohibited even if a different action on the same target is allowed.

13.3 OAuth Provider Integrations

The provider integration example is the concrete demonstration of the Arcade and WorkOS architecture described above. It does not require live provider credentials. Instead, it uses `StaticHttpClient` to stand in for WorkOS and Arcade responses while still exercising the same policy-engine and capability-minting path.

Run it with:

```
cargo run -p typesec-cli --example provider_integrations
```

The example composes three engines:

```
let engine = PolicyEngineBuilder::new()
    .add_engine(jwt_claims)
    .add_engine(workos)
    .add_engine(arcade)
    .strategy(CombineStrategy::PriorityOrder)
    .build();
```

The flow demonstrates three grants:

```
read org/acme              allowed from JWT claims
write project/proj_123     delegated to mocked WorkOS FGA
execute Gmail.ListEmails   delegated to mocked Arcade tool auth
```

The final step invokes a `ProtectedTool<CanExecute, _, _>`. That is the key piece of the example: after the provider engines allow the tool execution, Typesec still requires the local tool implementation to receive the typed execute capability before it can run.

13.4 Company Graph with Grust and Sail

The company graph example is the most complete integration. It models a company hierarchy:

```
employee:evelyn CEO
  <- REPORTS_TO employee:priya VP Engineering
      <- REPORTS_TO employee:marco Engineering Manager
          <- REPORTS_TO employee:nia Senior Software Engineer
          <- REPORTS_TO employee:omar Data Engineer
```

The policy assigns different graph-writing powers to different agents:

```
agent:executive-chief   writes executive data, company graph, and sensitive
network
agent:hr-onboarding     writes non-executive employees and reporting lines
agent:employee-nia      writes only Nia's public self-service profile
```

The Rust example uses the `grust` facade to build a backend-neutral property graph and to write the graph through the Sail adapter when a Sail SparkConnect server is available at `127.0.0.1:50051`. If Sail is not running, the example still demonstrates the Typesec decisions and prints the graph.

The import shape after switching to the published Grust crates is:

```
use grust::prelude::*;
```

That import pulls in the core graph types and, because the dependency enables the `sail` feature, the `SailConfig` and `SailGraphStore` adapter types.

13.5 Python LangChain-Style Tool Gating

The Python example is intentionally self-contained. It does not require LangChain to be installed and it does not call an LLM. Instead, it uses the shape of a LangChain tool wrapper:

```
tool call requested
  -> TypesecGate.allowed(subject, action, resource)
  -> cargo run -q -p typesec-cli -- check ...
  -> only then mutate the graph
```

This is a realistic integration boundary. Python agents can ask the Rust CLI for a policy decision before calling a tool. The Python code treats exit code `0` as allow and any nonzero exit code as blocked. A denied tool call raises `PermissionError` before the graph mutates.

This is not as strong as Rust's type-level API, because Python cannot enforce the same phantom-type proof. But it gives Python systems a practical, testable gate today.

14 Tests and Validation

The repository has tests at several levels.

Core unit tests check that capabilities carry the right subject and resource, that read and write capability types differ, and that denied engines do not mint capabilities.

Typestate tests check that authentication transitions from unauthenticated to authenticated and that empty subjects fail.

Agent tests check the full flow:

```
construct agent
authenticate
request capability
execute with capability
```

RBAC tests cover role grants, inheritance, unknown subjects, and denials. ODRL tests cover allowed purpose, wrong purpose, prohibition, and delegation for unknown subjects.

Integration tests in `crates/typesec-agent/tests/integration.rs` exercise the agent layer across more realistic scenarios. Provider integration tests in `crates/typesec-integrations/tests/provider_composition.rs` check the public WorkOS, Arcade, JWT, and capability-composition path. Python smoke tests in `tests/python/test_cli_policy.py` exercise the CLI as a policy oracle.

During today's final publishing pass, the merged repository was checked with:

```
cargo check --workspace
cargo test --workspace
cargo check -p typesec-cli --example provider_integrations
cargo run -q -p typesec-cli --example provider_integrations
```

When the Grust dependency was switched from a path dependency to published crates, the Grust/Sail example was checked separately:

```
cargo check --example company_graph_grust_sail
```

All passed.

15 What We Improved

The improvement notes in `docs/improvements.md` record a useful snapshot of the engineering work.

The workspace was upgraded to Rust 2024. Compiler failures were fixed across lattice tests, proc-macro parsing, examples, and doctests. Clippy was made to pass across all targets by tightening APIs, deriving defaults, documenting public ODRL fields, and applying simplifications.

The audit integration test was stabilized with a global capture subscriber for the test binary. That avoids a racy thread-local subscriber and makes the audit story more credible.

Python smoke tests were added around `typesec check`, giving non-Rust agent wrappers a low-friction test path.

The company graph examples were added to show Typesec at an application boundary, not just inside small unit tests. The Rust version proves the typed capability story with Grust. The Python version proves that the CLI can gate side-effecting tools in a LangChain-like shape.

The provider integration layer was added to show how Typesec fits under OAuth-based systems instead of replacing them. `JwtAuthenticator`, `JwtClaimsEngine`, `WorkOsFgaEngine`, and `ArcadeToolAuthEngine` all feed the same `PolicyEngine` boundary. `ProtectedTool` then demonstrates the local last mile: provider authorization is not merely observed; it becomes a typed capability required by the handler.

The new comparison document, `docs/typesec-and-auth-frameworks.md`, explains the strategic position: WorkOS handles enterprise identity and resource authorization, Arcade handles agent-oriented SaaS tool authorization, and Typesec makes the resulting local authority impossible to forget at the call site.

Finally, the Grust crates were switched from a local path dependency to published crates.io packages, making the example reproducible outside this machine.

16 Design Tradeoffs

Typesec deliberately separates runtime policy from compile-time proof. This is a tradeoff.

Putting all policy in types would be too rigid. Real policies come from YAML, databases, admin consoles, customer configuration, and external governance systems. They change without recompiling the application.

Leaving all policy at runtime is too easy to bypass. A forgotten guard can become a production vulnerability.

Typesec's compromise is:

```
runtime policy decides
typed capability proves the decision happened
protected APIs require the proof
```

Another tradeoff is that capabilities currently prove a decision at mint time. They do not yet model revocation, expiry, lease epochs, or policy-version binding. For short-lived operations, this is fine. For long-running agents, the system should grow epoch-bound or lease-bound capabilities.

The CLI is another pragmatic tradeoff. Rust code gets the strongest type-level story. Python code gets a subprocess oracle. That is weaker, but useful now. A future PyO3 crate could expose in-process policy checks, but the CLI boundary is easy to sandbox, inspect, and test.

The published Grust integration also records a naming tradeoff. The package is published as `grust-graph`, but it exposes the facade library as `grust`, so examples can keep the natural `use grust::prelude::*` shape.

The graph-policy example now uses two validation boundaries. Zod validates author-facing YAML and JSON before the policy graph is built. Grust's `GraphSchema` validates backend writes through `put_typed_graph`, so the same `Agent`, `Role`, `Employee`, `HAS_ROLE`, and `REPORTS_TO` model protects both policy loading and graph persistence.

The OAuth provider integration records a different tradeoff. Typesec should not be a token vault, a hosted IdP, or a full MCP runtime. The integration crate therefore uses small provider-specific

policy engines and injectable HTTP clients. That keeps provider decisions at the edge while preserving the core invariant: only an allow through `PolicyEngine` can mint the capability that local code requires.

The new `SecureValue` API is another compromise. It brings information-flow style labeled data into the Rust workspace without pretending to be a full language-level IFC system. The type-level `Join` lattice is intentionally small and sealed. That keeps the model reviewable, while still giving examples a real opaque value that can be transformed before explicit reveal or declassification.

17 Roadmap

The next phase should focus on proving the central promises more directly.

First, add compile-fail tests with `trybuild`. The most important tests are:

```
unauthenticated agents cannot request capabilities
actions cannot execute without capabilities
read capabilities cannot be passed where write capabilities are required
ordinary write cannot satisfy ai:exfiltrate
sensitive values cannot be unwrapped without reveal or declassify authority
```

Second, make generated policy code part of the examples. If `typesec generate` emits typed modules from an RBAC policy, downstream example code should compile against those generated types. Then a policy rename breaks code at compile time.

Third, refine deny, delegate, and constraint-failure semantics. Today ODRL uses delegation for no matching rule or failed permission constraints. That is useful for composition, but applications may want clearer distinctions in logs and errors.

Fourth, design capability expiry. A capability might carry a policy version, an epoch, or a lease deadline. Long-running agents should not keep using a proof forever after governance has changed.

Fifth, add `typesec check --json`. External agents need stable machine-readable answers:

```
{
  "verdict": "allow",
  "subject": "agent:data-pipeline",
  "action": "read",
  "resource": "reports/q1"
}
```

Sixth, deepen the Python story. The subprocess gate is enough to prove the boundary. A small Python package could wrap it cleanly. Later, a PyO3 binding could provide in-process checks for latency-sensitive integrations.

Seventh, expand the Grust example into an end-to-end backend demo that can run against a known local service profile. The current example gracefully skips Sail when it is not listening; a fuller demo could include setup instructions or a containerized path.

Eighth, add live provider smoke tests behind environment variables. The current WorkOS and Arcade tests use deterministic mocked HTTP clients, which is right for CI. A separate ignored test profile could verify a real WorkOS sandbox and a real Arcade project when credentials are present.

Ninth, extend `SecureValue` beyond the built-in four-label lattice. The current labels are `Public`, `Internal`, `Sensitive`, and `Secret`. Domain-specific deployments may want generated labels from policy files, capability-bound declassification reasons, or audited release records that carry policy version and purpose.

18 Conclusion

Typesec is a small system with a sharp idea: authorization should not only be a runtime answer. For agentic AI systems, authorization should also leave a typed trace in the program. If a function can write, read sensitive data, train a model, or exfiltrate content, that power should be visible in the function's inputs.

The repository now contains the first coherent version of that idea:

```
typed capabilities
opaque secure values
sealed permissions
resource abstractions
typestate authentication
RBAC policy evaluation
ODRL contextual policy and prohibitions
policy combinators
async secure agents
OAuth/OIDC provider integrations
WorkOS FGA and Arcade-style tool auth engines
CLI policy checks
Rust examples
Python tool-gating example
Grust/Sail graph integration
tests and documentation
```

The design is not finished, but it is real enough to build on. The next work is to make the compile-time guarantees more aggressively tested, make generated policy types part of ordinary workflows, connect the provider adapters to live sandbox environments, and give non-Rust agents cleaner ways to use the same security boundary.

That is the arc of today's build: from an idea about impossible-to-forget authorization, to a working Rust workspace, to examples that show how agent tools can be shaped around typed proof.